COLLEGE OF COMPUTING, INFORMATICS AND MATHEMATICS, UNIVERSITI TEKNOLOGI MARA SHAH ALAM

PROSOLVE NATIONAL 2025

FINALE ROUND PROBLEM SET

26 APRIL 2025

ORGANIZED BY:

ARTIFICIAL INTELLIGENCE SOCIETY (AIS),
COLLEGE OF COMPUTING, INFORMATICS AND MATHEMATICS, UiTM SHAH ALAM

CO-ORGANIZED BY:

GLOBALITE SOLUTION

PRIVATE AND CONFIDENTIAL

This Problem Set Contains 10 Problems (A-J)

Problem A

Regular Expression Matching

Given an input string s and a pattern p, implement regular expression matching with support for '.' and '*' where:

- '.' Matches any single character.
- '*' Matches zero or more of the preceding element.

The matching should cover the **entire** input string (not partial).

Example 1:

Input:	**Output:**
"aa" "a"	false

Explanation: "a" does not match the entire string "aa".

Example 2:

Input:	**Output:**
"aa" "a*"	true

Explanation: '*' means zero or more of the preceding elements, 'a'. Therefore, by repeating 'a' once, it becomes "aa".

Example 3:

Input:	**Output:**
"ab" ".*"	true

Explanation: ".*" means "zero or more (*) of any character (.)".

Constraints:

- 1 <= s.length <= 20
- 1 <= p.length <= 20
- s contains only lowercase English letters.
- p contains only lowercase English letters, '.', and '*'.
- It is guaranteed for each appearance of the character '*', there will be a previous valid character to match.

Problem B

Cracking the Safe

There is a safe protected by a password. The password is a sequence of n digits where each digit can be in the range [0, k - 1].

The safe has a peculiar way of checking the password. When you enter in a sequence, it checks the **most recent** n **digits** that were entered each time you type a digit.

- For example, the correct password is "345" and you enter in "012345":

  After typing 0, the most recent 3 digits is "0", which is incorrect. After typing 1, the most recent 3 digits is "01", which is incorrect. After typing 2, the most recent 3 digits is "012", which is incorrect. After typing 3, the most recent 3 digits is "123", which is incorrect. After typing 4, the most recent 3 digits is "234", which is incorrect. After typing 5, the most recent 3 digits is "345", which is correct and the safe unlocks.

Return *any string of* ***minimum length*** *that will unlock the safe* ***at some point*** *of entering it*.

Example 1:

Input:	**Output:**
1 2	"10"

Explanation: The password is a single digit, so enter each digit. "01" would also unlock the safe.

Example 2:

Input:	**Output:**
2 2	"01100"

Explanation: For each possible password:

"00" is typed in starting from the 4th digit. "01" is typed in starting from the 1st digit. "10" is typed in starting from the 3rd digit. "11" is typed in starting from the 2nd digit.
Thus "01100" will unlock the safe. "10011", and "11001" would also unlock the safe.

Constraints:

- 1 <= n <= 4
- 1 <= k <= 10
- 1 <= k^n <= 4096

Problem C

Count All Possible Routes

You are given an array of **distinct** positive integers locations where locations[i] represents the position of city i. You are also given integers start, finish and fuel representing the starting city, ending city, and the initial amount of fuel you have, respectively.

At each step, if you are at city i, you can pick any city j such that j != i and 0 <= j < locations.length and move to city j. Moving from city i to city j reduces the amount of fuel you have by |locations[i] - locations[j]|. Please notice that |x| denotes the absolute value of x.

Notice that fuel **cannot** become negative at any point in time, and that you are **allowed** to visit any city more than once (including start and finish).

Return *the count of all possible routes from* start *to* finish. Since the answer may be too large, return it modulo $10^9 + 7$.

Example 1:

Input:	**Output:**
[2,3,6,8,4] 1 3 5	4

Explanation: The following are all possible routes; each uses 5 units of fuel:

1 -> 3

1 -> 2 -> 3

1 -> 4 -> 3

1 -> 4 -> 2 -> 3

Example 2:

Input:	Output:
[4,3,1] 1 0 6	5

Explanation: The following are all possible routes:

1 -> 0, used fuel = 1

1 -> 2 -> 0, used fuel = 5

1 -> 2 -> 1 -> 0, used fuel = 5

1 -> 0 -> 1 -> 0, used fuel = 3

1 -> 0 -> 1 -> 0 -> 1 -> 0, used fuel = 5

Example 3:

Input:	Output:
[5, 2, 1] 0 2 3	0

Explanation: It is impossible to get from 0 to 2 using only 3 units of fuel since the shortest route needs 4 units of fuel.

Constraints:

- 2 <= locations.length <= 100
- 1 <= locations[i] <= 10^9
- All integers in locations are **distinct**.
- 0 <= start, finish < locations.length
- 1 <= fuel <= 200

Problem D

The Number of Good Subsets

You are given an integer array nums. We call a subset of nums **good** if its product can be represented as a product of one or more **distinct prime** numbers.

- For example, if nums = [1, 2, 3, 4]:
    - [2, 3], [1, 2, 3], and [1, 3] are **good** subsets with products 6 = 2*3, 6 = 2*3, and 3 = 3 respectively.
    - [1, 4] and [4] are not **good** subsets with products 4 = 2*2 and 4 = 2*2 respectively.

Return *the number of different **good** subsets in* nums ***modulo*** $10^9 + 7$.

A **subset** of nums is any array that can be obtained by deleting some (possibly none or all) elements from nums. Two subsets are different if and only if the chosen indices to delete are different.

Example 1:

Input:	**Output:**
[1,2,3,4]	6

Explanation: The good subsets are:

- [1,2]: product is 2, which is the product of distinct prime 2.

- [1,2,3]: product is 6, which is the product of distinct primes 2 and 3.

- [1,3]: product is 3, which is the product of distinct prime 3.

- [2]: product is 2, which is the product of distinct prime 2.

- [2,3]: product is 6, which is the product of distinct primes 2 and 3.

- [3]: product is 3, which is the product of distinct prime 3.

Example 2:

Input:	**Output:**
[4,2,3,15]	5

Explanation: The good subsets are:

- [2]: product is 2, which is the product of distinct prime 2.

- [2,3]: product is 6, which is the product of distinct primes 2 and 3.

- [2,15]: product is 30, which is the product of distinct primes 2, 3, and 5.

- [3]: product is 3, which is the product of distinct prime 3.

- [15]: product is 15, which is the product of distinct primes 3 and 5.

Constraints:

- 1 <= nums.length <= 10^5
- 1 <= nums[i] <= 30

Problem E

Buy Two Chocolates

You are given an integer array prices representing the prices of various chocolates in a store. You are also given a single integer money, which represents your initial amount of money.

You must buy **exactly** two chocolates in such a way that you still have some **non-negative** leftover money. You would like to minimize the sum of the prices of the two chocolates you buy.

Return *the amount of money you will have left over after buying the two chocolates*. If there is no way for you to buy two chocolates without ending up in debt, return money. Note that the leftover must be non-negative.

Example 1:

Input:	**Output:**
[1,2,2] 3	0

Explanation: Purchase the chocolates priced at 1 and 2 units respectively. You will have 3 - 3 = 0 units of money afterwards. Thus, we return 0.

Example 2:

Input:	**Output:**
[3,2,3] 3	3

Explanation: You cannot buy 2 chocolates without going in debt, so we return 3.

Constraints:

- 2 <= prices.length <= 50
- 1 <= prices[i] <= 100
- 1 <= money <= 100

Problem F

Longest String Chain

You are given an array of words where each word consists of lowercase English letters.

$word_A$ is a **predecessor** of $word_B$ if and only if we can insert **exactly one** letter anywhere in $word_A$ **without changing the order of the other characters** to make it equal to $word_B$.

- For example, "abc" is a **predecessor** of "abac", while "cba" is not a **predecessor** of "bcad".

A **word chain** is a sequence of words [$word_1$, $word_2$, ..., $word_k$] with k >= 1, where $word_1$ is a **predecessor** of $word_2$, $word_2$ is a **predecessor** of $word_3$, and so on. A single word is trivially a **word chain** with k == 1.

Return *the **length** of the **longest possible word chain** with words chosen from the given list of* words.

Example 1:

Input:	**Output:**
["a","b","ba","bca","bda","bdca"]	4

Explanation: One of the longest word chains is ["a","ba","bda","bdca"].

Example 2:

Input:	**Output:**
["xbc","pcxbcf","xb","cxbc","pcxbc"]	5

Explanation: All the words can be put in a word chain ["xb", "xbc", "cxbc", "pcxbc", "pcxbcf"].

Example 3:

Input:	Output:
["abcd","dbqca"]	1

Explanation: The trivial word chain ["abcd"] is one of the longest word chains.

["abcd","dbqca"] is not a valid word chain because the ordering of the letters is changed.

Constraints:

- 1 <= words.length <= 1000
- 1 <= words[i].length <= 16
- words[i] only consists of lowercase English letters.

Problem G

GCD Sort of an Array

You are given an integer array nums, and you can perform the following operation **any** number of times on nums:

- Swap the positions of two elements nums[i] and nums[j] if gcd(nums[i], nums[j]) > 1 where gcd(nums[i], nums[j]) is the **greatest common divisor** of nums[i] and nums[j].

Return true *if it is possible to sort* nums *in* ***non-decreasing*** *order using the above swap method, or* false *otherwise.*

Example 1:

Input:	**Output:**
[7,21,3]	true

Explanation: We can sort [7,21,3] by performing the following operations:

- Swap 7 and 21 because gcd(7,21) = 7. nums = [**21**,**7**,3]

- Swap 21 and 3 because gcd(21,3) = 3. nums = [**3**,7,**21**]

Example 2:

Input:	**Output:**
[5,2,6,2]	false

Explanation: It is impossible to sort the array because 5 cannot be swapped with any other element.

Example 3:

Input:	**Output:**
[10,5,9,3,15]	true

Explanation: We can sort [10,5,9,3,15] by performing the following operations:

- Swap 10 and 15 because gcd(10,15) = 5. nums = [**15**,5,9,3,**10**]

- Swap 15 and 3 because gcd(15,3) = 3. nums = [**3**,5,9,**15**,10]

- Swap 10 and 15 because gcd(10,15) = 5. nums = [3,5,9,**10**,**15**]

Constraints:

- $1 <= nums.length <= 3 * 10^4$
- $2 <= nums[i] <= 10^5$

Problem H

Minimum Deletions to Make Array Divisible

You are given two positive integer arrays nums and numsDivide. You can delete any number of elements from nums.

Return *the **minimum** number of deletions such that the **smallest** element in* nums ***divides*** *all the elements of* numsDivide. If this is not possible, return -1.

Note that an integer x divides y if y % x == 0.

Example 1:

Input:	**Output:**
[2,3,2,4,3] [9,6,9,3,15]	2

Explanation:

The smallest element in [2,3,2,4,3] is 2, which does not divide all the elements of numsDivide.

We use 2 deletions to delete the elements in nums that are equal to 2 which makes nums = [3,4,3].

The smallest element in [3,4,3] is 3, which divides all the elements of numsDivide. It can be shown that 2 is the minimum number of deletions needed.

Example 2:

Input:	**Output:**
[4,3,6] [8,2,6,10]	-1

Explanation:

We want the smallest element in nums to divide all the elements of numsDivide. There is no way to delete elements from nums to allow this.

Constraints:

- 1 <= nums.length, numsDivide.length <= 10^5
- 1 <= nums[i], numsDivide[i] <= 10^9

Problem I

Longest Chunked Palindrome Decomposition

You are given a string text. You should split it to k substrings ($subtext_1$, $subtext_2$, ..., $subtext_k$) such that:

- $subtext_i$ is a **non-empty** string.
- The concatenation of all the substrings is equal to text (i.e., $subtext_1 + subtext_2 + ... + subtext_k$ == text).
- $subtext_i$ == $subtext_{k - i + 1}$ for all valid values of i (i.e., 1 <= i <= k).

Return the largest possible value of k.

Example 1:

Input:	**Output:**
"ghiabcdefhelloadamhelloabcdefghi"	7

Explanation: We can split the string on "(ghi)(abcdef)(hello)(adam)(hello)(abcdef)(ghi)".

Example 2:

Input:	**Output:**
"merchant"	1

Explanation: We can split the string on "(merchant)".

Example 3:

Input:	Output:
"antaprezatepzapreanta"	11

Explanation: We can split the string on "(a)(nt)(a)(pre)(za)(tep)(za)(pre)(a)(nt)(a)".

Constraints:

- 1 <= text.length <= 1000
- text consists only of lowercase English characters.

Problem J

Longest Happy Prefix

A string is called a **happy prefix** if is a **non-empty** prefix which is also a suffix (excluding itself).

Given a string s, return *the **longest happy prefix** of* s. Return an empty string "" if no such prefix exists.

Example 1:

Input:	**Output:**
"level"	"l"

Explanation: s contains 4 prefix excluding itself ("l", "le", "lev", "leve"), and suffix ("l", "el", "vel", "evel"). The largest prefix which is also suffix is given by "l".

Example 2:

Input:	**Output:**
"ababab"	"abab"

Explanation: "abab" is the largest prefix which is also suffix. They can overlap in the original string.

Constraints:

- 1 <= s.length <= 10^5
- s contains only lowercase English letters.